# Week 12 Report

**Computational Statistics for Data Science · ISS507700**

Student ID: 112006234

May 17, 2026

*I have completed most of the work by myself, but I used AI tools to help with debugging, understanding some concepts, and checking for typos.*

## Q0 · Setup

```
rm(list = ls())

# load cars data
cars <- read.table(
  "auto-data.txt",
  header = FALSE,
  na.strings = "?",
  quote = "\"",
  col.names = c(
    "mpg", "cylinders", "displacement", "horsepower",
    "weight", "acceleration", "model_year", "origin", "car_name"
  )
)

# recreate cars_log
# log-transform all numeric variables except model_year and origin
cars_log <- data.frame(
  log.mpg. = log(cars$mpg),
  log.cylinders. = log(cars$cylinders),
  log.displacement. = log(cars$displacement),
  log.horsepower. = log(cars$horsepower),
  log.weight. = log(cars$weight),
  log.acceleration. = log(cars$acceleration),
  model_year = cars$model_year,
  origin = cars$origin
)

# remove rows with missing values
cars_log <- na.omit(cars_log)
```

## Q1 · Let's revisit the issue of multicollinearity of main effects.

**(a) Analyze the principal components of the four collinear variables**

**(i) Create a new data.frame of the four log-transformed variables with high multicollinearity (Give this smaller data frame an appropriate name – what might they jointly mean?)**

```
# these four variables are highly related to car size / engine power
car_size_vars <- cars_log[, c(
  "log.cylinders.",
  "log.displacement.",
  "log.horsepower.",
  "log.weight."
)]

# check correlation just to confirm multicollinearity
round(cor(car_size_vars), 2)
```

```
#>                 log.cylinders. log.displacement. log.horsepower. log.weight.
#> log.cylinders.            1.00              0.95            0.83        0.88
#> log.displacement.         0.95              1.00            0.87        0.94
#> log.horsepower.           0.83              0.87            1.00        0.87
#> log.weight.               0.88              0.94            0.87        1.00
```

The four variables are strongly correlated with each other, with correlations mostly above 0.80. This suggests multicollinearity, meaning they share a large amount of overlapping information. Conceptually, they can be grouped as car size or engine strength, because cars with more cylinders usually also have larger displacement, higher horsepower, and heavier weight.

**(ii) How much variance of the four variables is explained by their first principal component? (a summary of the prcomp() shows it, but try computing this from the eigenvalues alone)**

```
# run PCA
# scale.=TRUE uses the correlation matrix, which is safer when variables may have different rang
car_size_pca <- prcomp(
  car_size_vars,
  center = TRUE,
  scale. = TRUE
)

# show PCA summary
summary(car_size_pca)
```

```
#> Importance of components:
#>                           PC1     PC2     PC3     PC4
#> Standard deviation     1.9168 0.43316 0.32238 0.18489
#> Proportion of Variance 0.9186 0.04691 0.02598 0.00855
#> Cumulative Proportion  0.9186 0.96547 0.99145 1.00000
```

```r
# compute variance explained using eigenvalues only
car_size_eigen <- eigen(cor(car_size_vars))

eigenvalues <- car_size_eigen$values

pc1_variance <- eigenvalues[1] / sum(eigenvalues)

pc1_variance
```

```
#> [1] 0.9185647
```

```r
pc1_variance * 100
```

```
#> [1] 91.85647
```

> The first principal component explains about 91.86 percent of the total variance among the four variables.

This is very high, so one component captures most of the shared information from cylinders, displacement, horsepower, and weight. This supports using PC1 as a compact replacement for the four multicollinear variables.

**(iii) Looking at the values and valence (positiveness/negativeness) of the first principal component's eigenvector, what would you call the information captured by this component? (i.e., think what concept the first principal component captures or represents)**

```r
# look at the first principal component eigenvector / loading
pc1_loadings <- car_size_pca$rotation[, 1]

# flip sign only for easier interpretation:
# PCA signs can be positive or negative, but the meaning is the same.
if (mean(pc1_loadings) < 0) {
  pc1_loadings <- -pc1_loadings
}

round(pc1_loadings, 3)
```

```
#>    log.cylinders. log.displacement.   log.horsepower.       log.weight.
#>             0.498             0.512             0.486             0.504
```

> I would call it car size / engine strength

All four variables have positive and similar loadings, so PC1 represents the shared information among cylinders, displacement, horsepower, and weight. A higher PC1 score means the car is generally larger, heavier, and more powerful.

**(b) Let's revisit our regression analysis on cars_log:**

**(i) Store the scores of the first principal component as a new column of cars_log cars_log$new_column_name <- ...scores of PC1... Give this new column a name suitable for what it captures (see 1.a.i.)**

```
# store PC1 scores in cars_log
# flip sign if needed so larger values mean larger/heavier/powerful cars.
cars_log$car_size_pc1 <- car_size_pca$x[, 1]

if (mean(car_size_pca$rotation[, 1]) < 0) {
  cars_log$car_size_pc1 <- -cars_log$car_size_pc1
}
```

**(ii) Regress mpg over the column with PC1 scores (replacing cylinders, displacement, horsepower, and weight), as well as acceleration, model_year and origin**

```
# regress log mpg over PC1, acceleration, model year, and origin
regr_pc1 <- lm(
  log.mpg. ~ car_size_pc1 + log.acceleration. + model_year + factor(origin),
  data = cars_log
)

summary(regr_pc1)
```

```
#>
#> Call:
#> lm(formula = log.mpg. ~ car_size_pc1 + log.acceleration. + model_year +
#>     factor(origin), data = cars_log)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -0.51137 -0.06050 -0.00183  0.06322  0.46792
#>
#> Coefficients:
#>                   Estimate Std. Error t value Pr(>|t|)
#> (Intercept)        1.398114   0.166554   8.394 8.99e-16 ***
#> car_size_pc1      -0.145663   0.005057 -28.804  < 2e-16 ***
#> log.acceleration. -0.191482   0.041722  -4.589 6.02e-06 ***
#> model_year         0.029180   0.001810  16.122  < 2e-16 ***
#> factor(origin)2    0.008272   0.019636   0.421    0.674
#> factor(origin)3    0.019687   0.019395   1.015    0.311
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 0.1199 on 386 degrees of freedom
#> Multiple R-squared:  0.8772, Adjusted R-squared:  0.8756
#> F-statistic: 551.6 on 5 and 386 DF,  p-value: < 2.2e-16
```

The PC1 variable has a negative and highly significant coefficient. This means that larger/heavier/more powerful cars tend to have lower log mpg, which also means lower mpg in general, after controlling for acceleration, model year, and origin. Model year has a positive significant coefficient, meaning newer cars tend to have higher mpg. Acceleration is also significant in this model, while origin is not significant after controlling for the other variables.

**(iii) Try running the regression again over the same independent variables, but this time with everything standardized. How important is this new column relative to other columns?**

```
# standardized regression
# standardize continuous variables
# origin is categorical, so we keep it as a factor.
cars_std <- as.data.frame(scale(cars_log[, c(
  "log.mpg.",
  "car_size_pc1",
  "log.acceleration.",
  "model_year"
)]))

cars_std$origin <- cars_log$origin

regr_pc1_std <- lm(
  log.mpg. ~ car_size_pc1 + log.acceleration. + model_year + factor(origin),
  data = cars_std
)

summary(regr_pc1_std)
```

```
#>
#> Call:
#> lm(formula = log.mpg. ~ car_size_pc1 + log.acceleration. + model_year +
#>     factor(origin), data = cars_std)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -1.50385 -0.17791 -0.00538  0.18591  1.37608
#>
#> Coefficients:
#>                   Estimate Std. Error t value Pr(>|t|)
#> (Intercept)       -0.01589    0.02563  -0.620    0.536
#> car_size_pc1      -0.82112    0.02851 -28.804  < 2e-16 ***
#> log.acceleration. -0.10190    0.02220  -4.589 6.02e-06 ***
#> model_year         0.31611    0.01961  16.122  < 2e-16 ***
#> factor(origin)2    0.02433    0.05775   0.421    0.674
#> factor(origin)3    0.05790    0.05704   1.015    0.311
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
#>
#> Residual standard error: 0.3526 on 386 degrees of freedom
#> Multiple R-squared:  0.8772, Adjusted R-squared:  0.8756
#> F-statistic: 551.6 on 5 and 386 DF,  p-value: < 2.2e-16
```

```
# compare standardized coefficient sizes
# larger absolute values mean stronger relative importance
std_coefs <- coef(regr_pc1_std)

sort(abs(std_coefs[-1]), decreasing = TRUE)
```

```
#>      car_size_pc1        model_year log.acceleration.   factor(origin)3
#>        0.82111607        0.31611170        0.10189973        0.05789658
#>   factor(origin)2
#>        0.02432679
```

After standardizing the continuous variables, car size pc1 has the largest absolute coefficient, about 0.821. This means it is the most important predictor in this model compared with model year and acceleration.

The negative sign means that higher car size / engine strength is associated with lower mpg. Model year is the second strongest predictor, while acceleration has a smaller effect.

**Q2 · Please download the Excel data file security_questions.xlsx from Canvas. In your analysis, you can either try to read the data sheet from the Excel file directly from R (there might be a package for that!) or you can try to export the data sheet to a CSV file before reading it into R.**

---

```
library(readxl)

# load the 18 security question responses from the "data" sheet
security_data <- read_excel("security_questions.xlsx", sheet = "data")

# convert to data.frame, just to make base R functions easier to use
security_data <- as.data.frame(security_data)

# check structure
str(security_data)
```

```
#> 'data.frame':    405 obs. of  18 variables:
#>  $ Q1 : num  7 5 6 5 7 6 4 5 2 6 ...
#>  $ Q2 : num  5 5 6 5 7 5 2 5 2 6 ...
#>  $ Q3 : num  5 6 6 5 7 4 4 6 1 6 ...
#>  $ Q4 : num  7 6 6 5 7 5 7 4 3 5 ...
#>  $ Q5 : num  7 6 7 5 7 4 6 6 5 6 ...
#>  $ Q6 : num  4 5 6 5 4 4 1 5 2 4 ...
```

```
#>  $ Q7 : num  4 5 6 5 5 4 1 5 2 6 ...
#>  $ Q8 : num  7 7 6 5 7 5 6 5 2 6 ...
#>  $ Q9 : num  5 5 5 5 6 6 3 5 2 5 ...
#>  $ Q10: num  7 6 7 5 7 2 6 6 6 6 ...
#>  $ Q11: num  5 6 6 5 6 5 5 5 2 6 ...
#>  $ Q12: num  7 6 6 5 7 5 2 4 2 5 ...
#>  $ Q13: num  5 6 5 4 6 5 4 4 2 6 ...
#>  $ Q14: num  5 6 6 4 6 6 4 4 2 6 ...
#>  $ Q15: num  7 6 4 5 4 4 5 4 2 6 ...
#>  $ Q16: num  5 5 6 5 6 5 4 5 2 6 ...
#>  $ Q17: num  7 6 6 5 7 5 7 4 2 5 ...
#>  $ Q18: num  7 6 6 6 7 6 6 5 4 6 ...
```

```r
# the dataset should contain Q1 to Q18
security_items <- security_data[, paste0("Q", 1:18)]

# remove rows with missing values, if any
security_items <- na.omit(security_items)
```

**(a) How much variance did each extracted factor explain?**

```r
# run PCA on the 18 items
# since all questions are Likert-scale items, scale.=TRUE is still a good safe choice
security_pca <- prcomp(
  security_items,
  center = TRUE,
  scale. = TRUE
)

summary(security_pca)
```

```
#> Importance of components:
#>                           PC1     PC2     PC3     PC4     PC5     PC6     PC7
#> Standard deviation     3.0514 1.26346 1.07217 0.87291 0.82167 0.78209 0.70921
#> Proportion of Variance 0.5173 0.08869 0.06386 0.04233 0.03751 0.03398 0.02794
#> Cumulative Proportion  0.5173 0.60596 0.66982 0.71216 0.74966 0.78365 0.81159
#>                           PC8     PC9   PC10    PC11    PC12    PC13    PC14
#> Standard deviation     0.68431 0.67229 0.6206 0.59572 0.54891 0.54063 0.51200
#> Proportion of Variance 0.02602 0.02511 0.0214 0.01972 0.01674 0.01624 0.01456
#> Cumulative Proportion  0.83760 0.86271 0.8841 0.90383 0.92057 0.93681 0.95137
#>                           PC15   PC16   PC17   PC18
#> Standard deviation     0.48433 0.4801 0.4569 0.4489
#> Proportion of Variance 0.01303 0.0128 0.0116 0.0112
#> Cumulative Proportion  0.96440 0.9772 0.9888 1.0000
```

```r
# extract eigenvalues
security_eigenvalues <- security_pca$sdev^2
```

```r
# compute variance explained by each principal component
security_variance <- security_eigenvalues / sum(security_eigenvalues)

# make a readable table
security_variance_table <- data.frame(
  PC = paste0("PC", 1:length(security_eigenvalues)),
  Eigenvalue = round(security_eigenvalues, 3),
  Proportion_of_Variance = round(security_variance, 3),
  Cumulative_Variance = round(cumsum(security_variance), 3)
)

security_variance_table
```

```
#>       PC Eigenvalue Proportion_of_Variance Cumulative_Variance
#> 1    PC1      9.311                  0.517                0.517
#> 2    PC2      1.596                  0.089                0.606
#> 3    PC3      1.150                  0.064                0.670
#> 4    PC4      0.762                  0.042                0.712
#> 5    PC5      0.675                  0.038                0.750
#> 6    PC6      0.612                  0.034                0.784
#> 7    PC7      0.503                  0.028                0.812
#> 8    PC8      0.468                  0.026                0.838
#> 9    PC9      0.452                  0.025                0.863
#> 10  PC10      0.385                  0.021                0.884
#> 11  PC11      0.355                  0.020                0.904
#> 12  PC12      0.301                  0.017                0.921
#> 13  PC13      0.292                  0.016                0.937
#> 14  PC14      0.262                  0.015                0.951
#> 15  PC15      0.235                  0.013                0.964
#> 16  PC16      0.230                  0.013                0.977
#> 17  PC17      0.209                  0.012                0.989
#> 18  PC18      0.202                  0.011                1.000
```

PC1 explains the largest amount of variance, about 51.7 percent. PC2 explains about 8.9 percent, and PC3 explains about 6.4 percent.

After PC3, each component explains less than 5% of the variance. This suggests that most of the information in the 18 security questions is captured by the first few components, especially PC1.

**(b) How many dimensions would you retain, according to the two criteria we discussed? (Eigenvalue  1 and Scree Plot – can you show the screeplot with eigenvalue=1 threshhold?)**

```r
# criterion 1: Eigenvalue >= 1
num_pc_eigen1 <- sum(security_eigenvalues >= 1)

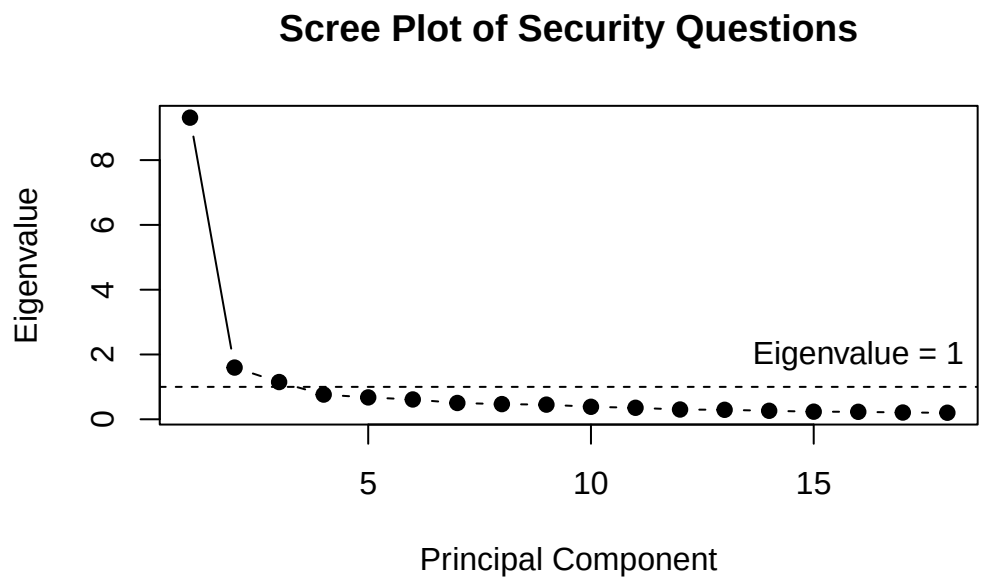
num_pc_eigen1
```

```
#> [1] 3
```

```
# criterion 2: Scree plot
plot(
  security_eigenvalues,
  type = "b",
  pch = 19,
  xlab = "Principal Component",
  ylab = "Eigenvalue",
  main = "Scree Plot of Security Questions"
)

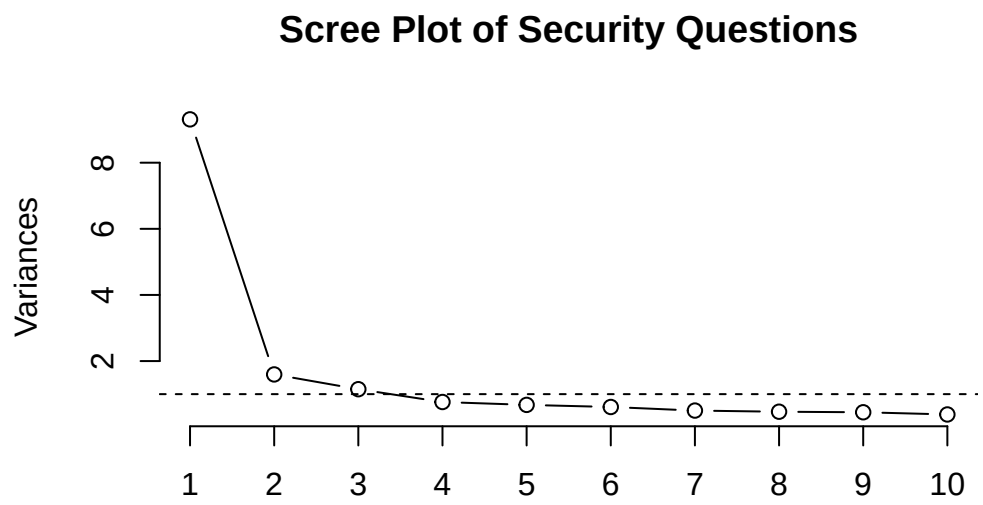
# add eigenvalue = 1 threshold line
abline(h = 1, lty = 2)

# label the threshold
text(
  x = length(security_eigenvalues) - 2,
  y = 1.1,
  labels = "Eigenvalue = 1",
  pos = 3
)
```




```
screeplot(
  security_pca,
  type = "lines",
  main = "Scree Plot of Security Questions"
)
```

```
abline(h = 1, lty = 2)
```




Using the eigenvalue $\geq 1$ criterion, I would retain 3 dimensions because only the first three components have eigenvalues greater than 1.

From the scree plot, there is a large drop from PC1 to PC2, and the curve starts flattening after around PC3. Therefore, both criteria suggest keeping about 3 principal components.

**(c) (ungraded) Can you interpret what any of the principal components mean? Try guessing the meaning of the first two or three PCs looking at the PC-vs-variable matrix**

```
round(security_pca$rotation[, 1:3], 3)
```

```
#>        PC1    PC2    PC3
#> Q1  -0.268 -0.110  0.002
#> Q2  -0.220 -0.011 -0.083
#> Q3  -0.251 -0.026 -0.084
#> Q4  -0.204  0.509 -0.101
#> Q5  -0.226 -0.025  0.506
#> Q6  -0.224 -0.083 -0.193
#> Q7  -0.215 -0.251 -0.302
#> Q8  -0.258  0.034  0.320
#> Q9  -0.237 -0.183 -0.190
#> Q10 -0.225 -0.078  0.497
#> Q11 -0.247 -0.207 -0.161
#> Q12 -0.207  0.505 -0.113
#> Q13 -0.233 -0.051 -0.079
```

```
#> Q14 -0.266 -0.079 -0.146
#> Q15 -0.231  0.008  0.310
#> Q16 -0.248 -0.161 -0.171
#> Q17 -0.202  0.526 -0.103
#> Q18 -0.264 -0.090  0.061
```

PC1 can be interpreted as overall perceived security, because almost all questions load in the same direction. The loadings are negative here, but PCA signs are arbitrary, so this still represents one common overall security dimension. PC2 seems related to transaction evidence and data integrity, since Q17, Q4, and Q12 have strong positive loadings. PC3 seems related to authentication and authenticity, because Q5, Q10, Q8, and Q15 load strongly on it.

These interpretations are only tentative, since PCA components are mathematical summaries and do not always have one clear real-world meaning.

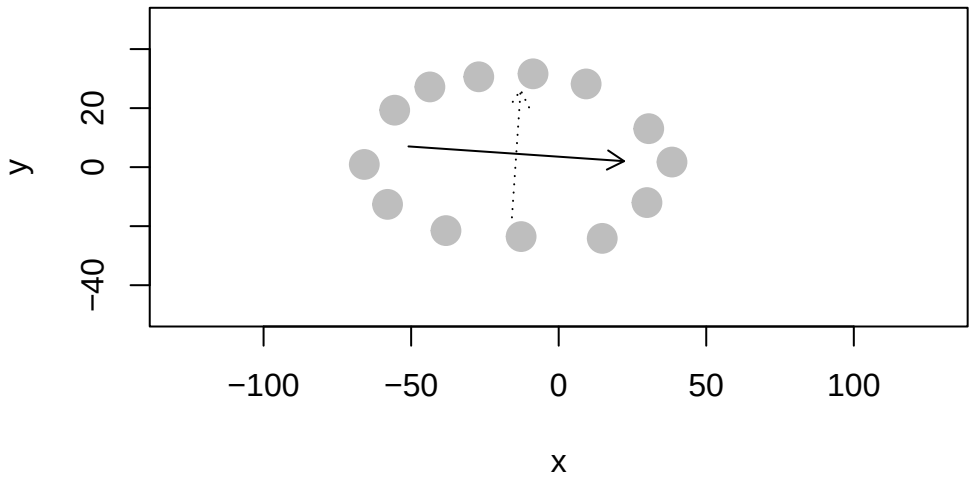
## Q3 · Let's simulate how principal components behave interactively: run the interactive_pca() function from the compstatslib package we have used earlier:

---

**(a) Create an oval shaped scatter plot of points that stretches in two directions – you should find that the principal component vectors point in the major and minor directions of variance (dispersion). Show this visualization.**

```
library(compstatslib)
#pca_oval <- interactive_pca()
#saveRDS(pca_oval, "pca_oval.rds")
# load the saved oval-shaped PCA points
pca_oval <- readRDS("pca_oval.rds")

# plot the PCA result
plot_pca(points = pca_oval$points)
```



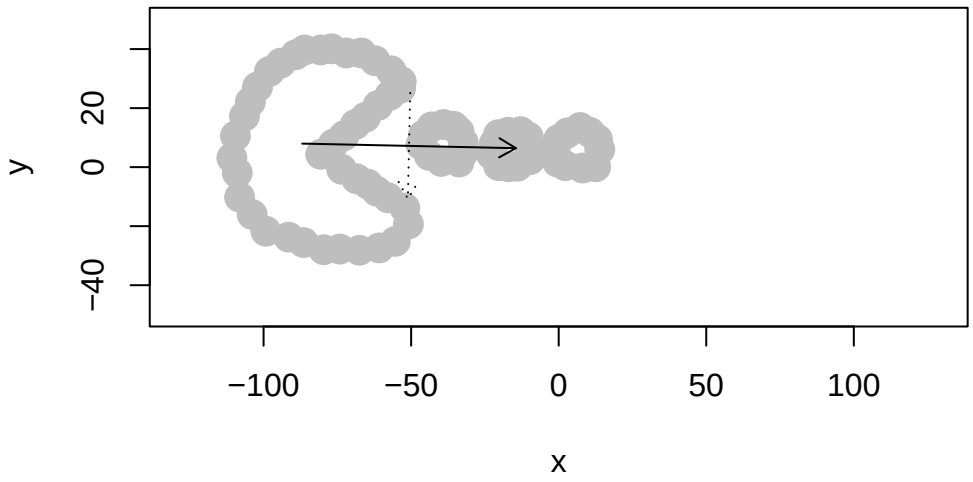

The first principal component follows the longer direction of the oval because this is the direction with the largest variance. The second principal component points in the shorter direction, where the data varies less. This matches the idea of PCA, which finds directions that explain the most spread in the data.

**(b) Can you create a scatterplot whose principal component vectors do NOT seem to match the major directions of variance? Show this visualization.**

```
library(compstatslib)
#pca_weird <- interactive_pca()
#saveRDS(pca_weird, "pca_weird.rds")

# load the saved non-oval / unusual PCA points
pca_weird <- readRDS("pca_weird.rds")

# plot the PCA result
plot_pca(points = pca_weird$points)
```




The overall point pattern is curved and irregular rather than following one simple straight direction. The principal component vectors still capture the overall linear spread of the data, but they do not follow the actual curved Pac-Man-like structure very well (approved by the Professor). This shows that PCA can be less suitable when the main pattern in the data is non-linear.